Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence

JUNAN ZHANG, School of Computer Science, Fudan University, Shanghai, China
KAIFENG HUANG, School of Computer Science and Technology, Tongji University, Shanghai, China
YIHENG HUANG, BIHUAN CHEN, RUISI WANG, CHONG WANG, and XIN PENG,
School of Computer Science, Fudan University, Shanghai, China

Open source software (OSS) supply chain enlarges the attack surface of a software system, which makes package registries attractive targets for attacks. Recently, multiple package registries have received intensified attacks with malicious packages. Of those package registries, NPM and PyPI are two of the most severe victims. Existing malicious package detectors are developed with features from a list of packages of the same ecosystem and deployed within the same ecosystem exclusively, which is infeasible to utilize the knowledge of a new malicious NPM package detected recently to detect the new malicious package in PyPI. Moreover, existing detectors lack support to model malicious behavior of OSS packages in a sequential way.

To address the two limitations, we propose a single detection model using malicious behavior sequence, named Cerebro, to detect malicious packages in NPM and PyPI. We curate a feature set based on a high-level abstraction of malicious behavior to enable multi-lingual knowledge fusing. We organize extracted features into a behavior sequence to model sequential malicious behavior. We fine-tune the pre-trained language model to understand the semantics of malicious behavior. Extensive evaluation has demonstrated the effectiveness of Cerebro over the state-of-the-art as well as the practically acceptable efficiency. Cerebro has detected 683 and 799 new malicious packages in PyPI and NPM, and received 707 thank letters from the official PyPI and NPM teams.

CCS Concepts: • **Security and privacy → Software security engineering**; • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering → Software libraries and repositories**;

J. Zhang and K. Huang contributed equally as first authors.

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62332005, 62372114 and 62402342) and Shanghai Sailing Program (No. 24YF2749500).

Authors' Contact Information: Junan Zhang, School of Computer Science, Fudan University, Shanghai, China; e-mail: 19307110280@fudan.edu.cn; Kaifeng Huang (corresponding author), School of Computer Science and Technology, Tongji University, Shanghai, China; e-mail: kaifengh@tongji.edu.cn; Yiheng Huang, School of Computer Science, Fudan University, Shanghai, China; e-mail: huangyh1998@foxmail.com; Bihuan Chen (corresponding author), School of Computer Science, Fudan University, Shanghai, China; e-mail: bhchen@fudan.edu.cn; Ruisi Wang, School of Computer Science, Fudan University, Shanghai, China; e-mail: sunbk201gm@gmail.com; Chong Wang, School of Computer Science, Fudan University, Shanghai, China; e-mail: wangchong20@fudan.edu.cn; Xin Peng, School of Computer Science, Fudan University, Shanghai, China; e-mail: pengxin@fudan.edu.cn.

ACM 1557-7392/2025/4-ART104
https://doi.org/10.1145/3705304

Additional Key Words and Phrases: malicious package detection, open source packages, behavior sequence modeling, large language models

ACM Reference format:
Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 104 (April 2025), 28 pages.
https://doi.org/10.1145/3705304

1 Introduction

As the adoption of **open source software (OSS)** continues to grow, the security concerns about OSS supply chain have attracted increased attention [20, 29, 42, 72, 77]. The frequency and impact of OSS supply chain attacks have reached unprecedented levels. According to Sonatype [74], there has been an astonishing average annual increase of 742% in OSS supply chain attacks over the past 3 years. According to Gartner's prediction [22], 45% of organizations worldwide will have experienced OSS supply chain attacks by 2025. This alarming trend can be attributed to the expanded attack surface through the OSS supply chain. For example, installing an NPM package introduces an average of 79 transitively dependent packages and 39 maintainers which can be exploited; and some popular packages influence more than 100,000 packages and thus become attractive targets for attacks [96].

Problem. One of the OSS supply chain attacks is to inject malicious code into packages hosted in popular package registries. Package registries NPM and PyPI have been flooded with malicious packages, as revealed by recent reports [34, 45, 61, 71, 73]. For example, the PyPI team removed over 12,000 packages in 2022, which were mostly malware [34]; and the Sonatype team caught 422 malicious NPM packages and 58 malicious PyPI packages in December 2022 [73], mostly data exfiltration through typosquatting or dependency confusion attacks. An alarming incident occurred in December 2022 was the dependency confusion attack on PyTorch [66]. The attack exploited the fact that the nightly built version of PyTorch depended on a package named "torchtriton" from the PyTorch nightly package index. However, a malicious package with the same name was uploaded to PyPI. Since PyPI takes precedence over the PyTorch nightly package index, the malicious "torchtriton" was installed instead of the legitimate version. The prevalence of malicious packages in NPM and PyPI calls for practically effective malicious package detection system.

Existing Approaches. Several approaches have been proposed to detect malicious packages in NPM and PyPI. They can be classified into rule-based [12, 51, 80, 83, 87, 88, 94], unsupervised learning [21, 47, 58], and supervised learning approaches [15, 16, 43, 57, 70, 90]. Rule-based approaches often rely on pre-defined rules about package metadata (e.g., package name) and suspicious imports and method calls. They often incur high false positives, which is far from reaching practical demands [85]. Learning-based approaches capture malicious behavior as a set of discrete features. They overlook the sequential nature of malicious behavior which is usually composed of a sequence of suspicious activities, hindering the practical effectiveness. Moreover, except for [12, 51, 80], these approaches are designed and evaluated for one ecosystem (i.e., either NPM or PyPI). OSS Detect Backdoor [51] and Taylor et al. [80] adopt lightweight rules to support different ecosystems, while Duan et al. [12] use heavyweight rules that require both static and dynamic program analysis.

Limitations. We summarize two limitations that hinder the effectiveness of existing approaches. The first limitation is that the knowledge of malicious packages from different ecosystems is not sufficiently leveraged. Although there are evident clues of attackers transcribing malicious packages and spreading them across multiple ecosystems [65], there has been limited action in addressing

this evolving threat landscape. Moreover, the NPM and PyPI teams do not publicly release the entire collection of malicious packages for preventing potential misuse or exploitation, and hence the publicly available datasets of malicious NPM and PyPI packages [12, 59] are small in size, which are significantly smaller than the reported ones [34, 73]. The malicious package detection may be less effective when the available malicious package data are limited.

The second limitation is that sequential knowledge is missing in existing approaches. Malicious packages typically conduct a sequence of suspicious activities to achieve an attack. However, existing rule-based and learning-based approaches fail to consider the sequential nature of malicious behavior. Consequently, false positives and negatives may arise due to imprecise modeling.

Our Approach. To address these two limitations, the challenges become (1) *how to leverage the knowledge of malicious packages from different ecosystems in a unified way such that multi-lingual malicious package detection can be feasible* and (2) *how to model malicious behavior in a sequential way such that maliciousness can be precisely captured.* To that end, we propose a single model, named Cerebro, to detect malicious packages in NPM and PyPI using malicious behavior sequences.

Cerebro has three key components which are the feature extractor, behavior sequence generator, and maliciousness classifier. The feature extractor employs static analysis to extract a set of 16 features. We curate this feature set based on a high-level abstraction of malicious behavior, which is language independent. Thus, it enables multi-lingual knowledge fusing across NPM and PyPI and solves the first challenge. Notably, Cerebro is focused solely on static features and does not consider metadata or dynamic features, as it aims to strike a balance between effectiveness and efficiency.

The behavior sequence generator produces a behavior sequence of a package by organizing extracted features based on their likelihood of execution and their sequential order in execution, which solves the second challenge. We determine the execution likelihood according to the time phase of execution (i.e., install-time, import-time or runtime), and we determine the sequential order according to generated call graph.

The maliciousness classifier employs a fine-tuned language model [37, 50, 67] on the generated behavior sequence to determine whether a package is malicious or not. We transform the behavior sequence into a textual description to ease the semantic understanding of malicious behavior. We fine-tune the model on NPM and/or PyPI packages into a binary classifier.

Evaluation. To evaluate the effectiveness and efficiency of Cerebro, we conduct extensive experiments on a dataset of 2,675 malicious and 7,391 benign NPM and PyPI packages. Our evaluation results have demonstrated that Cerebro outperforms the state-of-the-art by an average of 10.0% in precision and 7.4% in recall in the mono-lingual scenario (i.e., train and test on the same ecosystem), and by 9.9% in precision and 8.9% in recall in the bi-lingual scenario (i.e., train on two ecosystems and test on one of the two ecosystems). Cerebro takes an average of 10.5 seconds to analyze a package, which is practically efficient. Further, we conduct an ablation study to validate the contribution of behavior sequence.

To evaluate the usefulness of Cerebro in practice, we run Cerebro on the newly published packages in PyPI and NPM over 8 and 7 months. From 923,638 newly published package versions, we detect 5,976 potentially malicious package versions. After our manual confirmation, we detect 683 malicious PyPI package versions and 799 malicious NPM package versions. We report these detected malicious package versions to the official PyPI and NPM teams. All these malicious package versions have been removed by the official teams; 775 of them have already been removed before our report, and hence we receive 707 thank letters for the remaining ones.

Contributions. Our work makes following contributions.

—We proposed and implemented a single model using malicious behavior sequence, named Cerebro, to detect malicious packages in NPM and PyPI.

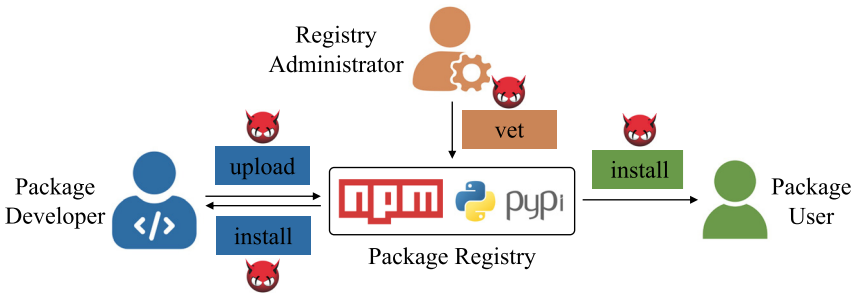


Fig. 1. Threats in the package registry ecosystem.

—We conducted extensive experiments to demonstrate the effectiveness and efficiency of Cerebro.
—We detected 683 and 799 new malicious packages in PyPI and NPM, and received 707 thank letters from the official team of PyPI and NPM.

2 Threat Model

We discuss the stakeholders and their activities involved in the development and distribution of OSS. Different from existing work [42], our threat model is particularly focused on the threats of using package registries, i.e., NPM [24] and PyPI [19]. NPM and PyPI are widely recognized as the primary package registries for hosting Python and JavaScript packages, respectively. To facilitate package management, PyPI is seamlessly integrated with the *pip* package manager, which is the default package manager for Python. Similarly, NPM is closely integrated with the *npm* package manager. As shown in Figure 1, **package developers (PDs)** develop and maintain packages, and use the package manager to upload their packages to the package registry. **Registry administrators (RAs)** then vet the uploaded packages and determine whether they should be published. Once published, packages become publicly available on the package registry. **Package users (PUs)** then leverage the package manager to conveniently download and install desired packages for use in their own projects. We analyze the threats in these package management steps, which involve three key stakeholders (i.e., PDs, RAs, and PUs).

2.1 Threats in Developing Packages

The threats in developing packages allow the injection of malicious code into packages. We summarize two key threats.

Weak/Compromised Credentials. Various tools (e.g., version control systems and build systems) are used during the development of packages. Hence, account hijacking may happen due to weak credentials [39], or compromised credentials by exploiting vulnerabilities in these tools [30, 41]. As a result, attackers can gain access to PDs' accounts and hence have the privilege to inject malicious code.

Weak Governance in Collaborative Development. Packages are collaboratively developed by OSS community with many PDs. However, the governance of PDs is weak. First, malicious contributors can fool the development team of an existing package. They may first pretend to be benign to gain trust by committing useful features and then secretly commit malicious code, or submit pull requests that fix bugs or add useful features but also include additional malicious code [23]. Second, benign PDs of an existing package may add malicious PDs into the team [75] or become malicious PDs by social engineering tactics, or the ownership is transferred to malicious PDs [8]. As a result, malicious PDs have the full right to inject malicious code and publish malicious packages. Third, instead of infecting existing packages, attackers may first publish a benign and useful package, wait

until it is used, and then update it to include malicious code [56], or launch squatting attacks to inject malicious code into a new package whose name is similar to a popular and benign package [87].

2.2 Threats in Vetting Packages

The threats in vetting packages allow malicious packages to be publicly available to PUs. We summarize two key threats.

Ineffective/Insufficient Package Vetting. A vetting system is often employed by RAs to first automatically identify suspicious packages and then manually triage them. However, the vetting system lacks effectiveness, which allows malicious packages to evade detection and be published. This is evidenced by the recent report from Snyk [10], which has documented around 6,800 malicious packages on PyPI and NPM since the beginning of 2023. One major contributing factor is the overwhelming flood of suspicious packages and the limited resources and budgets for RAs. For example, PyPI had one person on-call to hold back weekend malware rush [9]. A recent interview with PyPI's RAs also confirms this dilemma [85]. Consequently, the manual triage in the vetting system is not sufficient, allowing malicious packages to bypass the system.

Insecure Governance of RAs. As package registries are often maintained by the OSS community, not all RAs can be blindly trusted. Attackers can disguise themselves as trustworthy developers to gain RAs' trust; and they may employ social engineering tactics to gain control of RAs' accounts. As a result, such malicious RAs have the right to grant and publish malicious packages to registries.

2.3 Threats in Using Packages

The threats in using packages allow malicious behaviors to be potentially triggered. We summarize two key threats.

Weak Awareness of Security. When PUs (including application developers and PDs) input the package name, the package manager searches and retrieves the corresponding package from the package registry. Unfortunately, this process is susceptible to various attacks. Attackers often exploit techniques such as typosquatting [80], combosquatting [87], and dependency confusion [4] to deceive PUs into mistakenly installing malicious packages. Attackers may also employ search engine optimization poisoning or phishing techniques [35] to advertise their packages on package registries, increasing the risk of unintended installation.

Weakness in Automated Dependency Resolution. Package managers employ automated dependency resolution algorithms to select package versions when updating package versions and installing transitive dependencies. As a result, malicious package versions can be silently installed. This places a long-standing burden on PUs, as they must consistently monitor potential compromises among package updates [96].

3 Preliminary and Motivation

We present preliminary information on triggering malicious behavior and then show two motivating examples.

3.1 Triggering Malicious Behavior

Once malicious code is present in an application's OSS supply chain (i.e., its direct and transitive dependencies) in NPM and PyPI, malicious behavior can be triggered in three scenarios.

Install-Time Execution. Malicious code is often contained in install scripts that are automatically executed during package installation [59]. For PyPI packages, *pip* automatically executes the `setup.py` script present at the root of the package during installation. This script is responsible for performing any necessary preparation or configuration required for the installation. Similarly, NPM introduces a mechanism that utilizes *scripts* property in the `package.json` [11]. This property

Table 1. Feature Types Used in Existing Literature

Feature Type	Description	Example Literature
Metadata feature	Package name, package size, uploader profile, and so on	Zimmermann et al. [96], Taylor et al. [80], Vu et al. [87], Zahan et al. [94]
Syntactic feature	Elements and structures within the source code	Wentao et al. [90], Vu et al. [84], Vu et al. [86], Ladisa et al. [43]
Semantic feature	Features that have explicit semantic meanings	Garrett et al. [21], Fang et al. [14]
Implicit feature	Implicit features vectorized from program dependence graph, abstract syntax trees, and so on	Ohm et al. [58]
Dynamic feature	Features captured during runtime execution	Duan et al. [12], Ohm et al. [60]
Hybrid feature	Mixture of the single feature types	Liang et al. [47], Duan et al. [12], Sejfia and Schäfer [70], Ohm et al. [57]

allows developers to specify certain keys, such as *preinstall* and *postinstall*, to indicate the paths of scripts to be automatically executed. In this scenario, attackers inject the malicious payload into install scripts. The malicious payload is executed without requiring any additional action as long as the infected package is installed. This scenario provides attackers with the highest probability of successfully carrying out attacks.

Import-Time Execution. Malicious code can also be executed when a package module is imported. This is achieved in Python/JavaScript by injecting malicious code into the initialization file `__init__.py`/`index.js`, which is executed by default when the interpreter executes *import*/*require* statements. Further, the interpreter continues to load the imported module file and execute the code at the global scope. The code at the global scope typically refers to statements that exist outside any method or class declaration. For the ease of presentation, we refer to the code at the global scope as the implicit "main" method. Attackers take advantage of this module import mechanism by injecting malicious code into this implicit "main" method.

Runtime Execution. Malicious code can also be executed at runtime during the normal control flow of applications. This is achieved by including the malicious code in a legitimate method that is unlikely to raise suspicion while hoping that the infected method will be invoked by applications. This scenario provides the largest attack surface, as attackers can inject malicious code into any package method. However, it achieves the lowest probability of successfully carrying out attacks, as the infected method has a low chance of being called.

3.2 Literature Survey on Malicious PyPI and NPM Package Features

We conducted a comprehensive literature survey to understand malicious features in PyPI and NPM packages. We searched for related academic papers on Google Scholar using the keywords "malicious," "PyPI," and "NPM." We also employed a snowballing process to include referenced and cited papers. In total, we obtained 16 papers including long technical papers and short papers. We categorized the features used in those papers into six types, i.e., metadata feature, syntactic feature, semantic feature, implicit feature, dynamic feature, and hybrid feature, presented in Table 1. Additionally, we conducted a deeper comparison with the related literature focusing on semantic and hybrid features. We excluded metadata features due to their lack of generalizability and adaptability to different package registries with varying metadata. Syntactic and implicit features were excluded because their results are often uninterpretable. Dynamic features were also excluded due to their significant computational overhead and time-consuming nature.

Based on above literature, we summarize the malicious features into six dimensions based on a high-level abstraction of malicious behavior, i.e., metadata, information reading, data transmission,

Table 2. Feature Set to Model Malicious Behavior

Dimension	Feature Description	Cerebro	Amalfi [70]	MalOSS [12]	PPD [47]	Garrett et al. [21]	Ohm et al. [57]	Fang et al. [14]
Metadata	M1: suspicious package name	○	○	●	●	○	●	○
	M2: suspicious maintainer	○	○	●	○	○	○	○
	M3: malicious dependencies	○	○	●	○	○	○	○
	M4: abnormal publish time	○	●	●	○	○	○	○
	M5: contain package install script	○	●	●	●	○	●	○
	M6: contain executable file	○	●	●	○	○	○	○
Information reading	R1: import OS module	●	●	○	○	○	○	●
	R2: use OS module call	●	○	●	●	○	○	●
	R3: import file system module	●	●	○	○	○	●	●
	R4: use file system module call	●	○	●	●	●	○	●
	R5: read sensitive information	●	●	●	●	○	●	●
Data transmission	D1: import network module	●	●	○	○	○	●	●
	D2: use network module call	●	○	●	●	●	○	●
	D3: use URL	●	○	●	●	○	●	●
Encoding	E1: import encoding module	●	●	○	○	○	○	●
	E2: use encoding module call	●	○	○	●	○	●	●
	E3: use base64 string	●	○	○	●	○	●	○
	E4: use long string	●	○	○	●	○	○	○
Payload execution	P1: import process module	●	●	○	○	○	●	●
	P2: use process module call	●	○	●	○	●	○	●
	P3: use bash script	●	○	○	●	○	○	○
	P4: evaluate code at runtime	●	●	●	○	●	●	○
Dynamic	/	○	○	●	○	○	○	○

●, Supported; ○, Unsupported.

encoding, and payload execution, which is presented in Table 2. We can observe the existing approaches have varying implementation on the metadata and semantic features. Metadata, information reading, and data transmission are the most frequently mentioned dimensions, followed by encoding and payload execution.

3.3 Motivating Examples

Example 1: Malicious Packages from PyPI and NPM Share Similar Malicious Behavior. Figure 2 shows the snippets of malicious PyPI package *dpp_client-1.0.3* and NPM package *botbait-1.0.0*, which are taken from Backstabber's Knife Collection dataset [59]. The package *dpp_client-1.0.3* gathers sensitive information, including current working directory, user name, and hostname, via executing CLI commands (e.g., *pwd*, *whoami*, and *hostname*) in `setup.py` (Lines 4 to 9). Similarly, the package *botbait-1.0.0* collects sensitive information about process's version, platform, and architecture in `index.js` (Lines 2 to 7). Both packages send the sensitive information to remote servers using different libraries, i.e., *requests* in `setup.py` (Line 12) and *http* in `index.js` (Lines 14 to 17). Although being implemented in different languages with distinct syntax, these packages exhibit similar malicious behavior, involving the read of sensitive information and the calling of

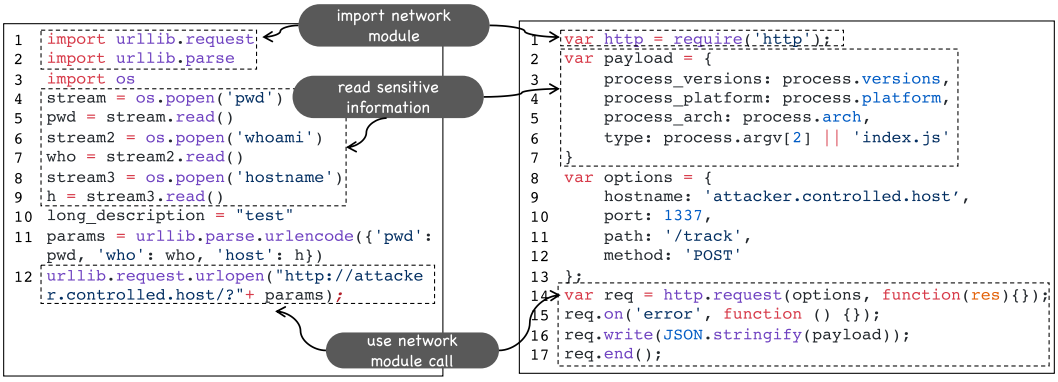


Fig. 2. Malicious packages from PyPI and NPM that share similar malicious behavior.

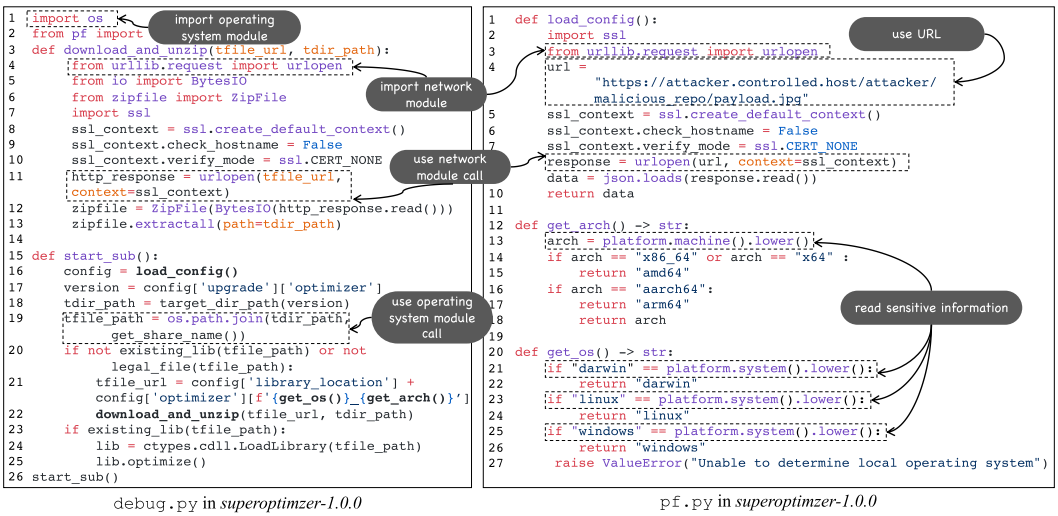


Fig. 3. A malicious package from PyPI with a malicious behavior sequence.

network operations. Sonatype also found that some malicious PyPI and NPM packages shared the same author, contained the identical malicious code, and used the same offending URL to fetch content from the remote [71].

Therefore, this opens up new opportunities for fusing the knowledge of malicious packages from different ecosystems at a high-level abstraction and for designing a unified detection system that supports different ecosystems. Such a unified detection system can partially address the challenge of limited dataset of malicious packages. It can also allow us to identify unknown malicious packages that may have already been encountered in other package registries.

Example 2: A Malicious Package Has a Malicious Behavior Sequence. Figure 3 shows the snippet of a malicious PyPI package *superoptimzer-1.0.0*. The malicious behavior sequence is located in `debug.py` and `pf.py`, and is triggered upon the import of `debug.py`. When `debug.py` is imported, the method `start_sub` is invoked in `debug.py` (Line 26). In `start_sub`, there are several inter-procedural calls that carry out the attack, i.e., `load_config` (Line 16), `get_os` (Line 21), `get_arch`

(Line 21), and `download_and_unzip` (Line 22). In `load_config` (Lines 1 to 10 in `pf.py`), two suspicious activities, including using URL and using network call, download the payload (in a json format) from a remote server to `config`. Then, `config` is parsed to obtain `version`, `tdir_path` and `tfile_path` (Lines 17 to 19 in `debug.py`). In `get_os` (Lines 20 to 27 in `pf.py`) and `get_arch` (Lines 12 to 18 in `pf.py`), sensitive information about the underlying OS and architecture is read. Then, a target URL `tfile_url` is obtained by concatenating `config['library_location']` and `config['optimizer']` (Line 21 in `debug.py`). Finally, in `download_and_unzip` (Lines 3 to 13 in `debug.py`), a network call is used to download the payload from `tfile_url`. We can observe that the malicious behavior is often composed of a sequence of suspicious activities.

Therefore, the modeling of malicious behavior sequence plays a crucial role in the accuracy of malicious package detection system. However, the state-of-the-art detection systems model malicious behavior as discrete features and do not consider the sequential nature of malicious behavior.

4 Methodology

We first introduce the overview of our approach, then elaborate each step of our approach in detail, and finally present the implementation of our approach.

4.1 Approach Overview

The goal of our approach is to support the analysis of NPM and PyPI packages with a unified model by leveraging the bi-lingual knowledge of malicious packages from NPM and PyPI as well as their sequential malicious behavior knowledge. To achieve this, we propose Cerebro, a system designed to detect malicious packages in NPM and PyPI ecosystems. The approach overview of Cerebro is shown in Figure 4. Overall, it consists of three key components, i.e., feature extractor (see Section 4.2), behavior sequence generator (see Section 4.3), and maliciousness classifier (see Section 4.4).

In the offline training phase, Cerebro takes inputs as a set of malicious and benign NPM and PyPI packages, uses feature extractor and behavior sequence generator to prepare the fine-tuning inputs, and fine-tunes a maliciousness classifier as the output. Similarly, in the online prediction phase, Cerebro takes an NPM or PyPI package as the input, uses feature extractor and behavior sequence generator to prepare the prediction input, and uses the maliciousness classifier to determine whether the package is malicious or benign.

Specifically, the feature extractor component extracts features from each source code file via static analysis. It enables bi-lingual knowledge fusing across NPM and PyPI through a set of language-independent features derived from a high-level abstraction of malicious behavior. Then, the behavior sequence generator component generates behavior sequence for each package based on extracted features. It captures the sequential relation among extracted features by call graph traversal. Finally, the maliciousness classifier component fine-tunes a pre-trained language model with behavior sequences into a binary classifier for malicious package detection.

Our malicious package detection system addresses the threats in Section 2 as it trusts none of the stakeholders and prevents malicious packages from being published into package registries and thus from being used as well.

4.2 Feature Extractor

The details of our features are presented in Table 2. Each feature has a corresponding ID and a textual description (e.g., R5: read sensitive information). We use common words to describe the feature to make use of the full capability of the language model in the maliciousness classifier (see Section 4.4). Due to space limitation, we compare our feature set with only state-of-the-art

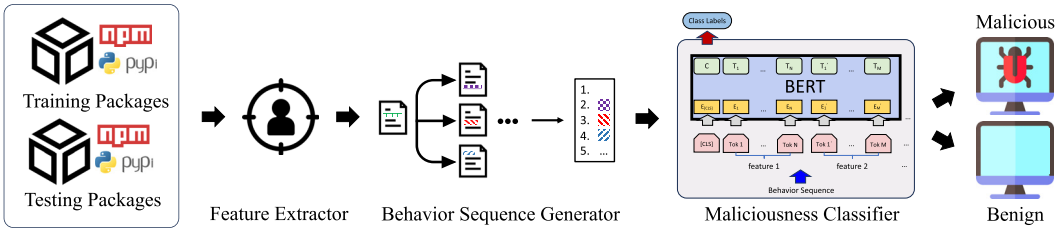


Fig. 4. An approach overview of Cerebro for detecting malicious NPM and PyPI packages.

rule-based approach MalOSS [12], unsupervised approaches in the work of Garrett et al. [21] and PPD [47], and supervised approaches Amalfi [70] and the work of Ohm et al. [57]. Notice that different approaches may have different implementations for a rule. For example, MalOSS considers a package name suspicious if it is similar to popular ones in the same registry, or if it is the same as popular packages in different registries, but with different authors. Differently, PPD determines a suspicious package name simply based on the package name's Levenshtein distance.

Metadata. The metadata includes package name (M1), maintainers (M2), dependencies (M3), publish time (M4), and the existence of specific file types (M5 and M6). These metadata features only provide the abstraction for appearance characteristics without looking into code-level behavior. Besides, analyzing metadata features relies on assumptions that may not hold in some circumstances. For example, information about popular packages may not exist universally in different package registries, or it is difficult to identify suspicious maintainers as prior knowledge. In order to make Cerebro more universally applicable, Cerebro does not leverage any metadata feature, but only uses features that can be extracted from code. In this way, Cerebro operates independently of specific registry characteristics and enhances its capability to detect malicious behavior across different package registry sources.

Information Reading. Attackers often attempt to read sensitive information (R5), including personal information (e.g., account details, passwords, crypto wallets, and credit card information) as well as machine-related information (e.g., runtime environment and machine name). They either steal such information, or use such information to carry out the attack. Besides, to effectively read sensitive information, attackers often leverage utility libraries provided by OS and file system. To detect such behavior, we focus on identifying the utilization of these libraries in the package, including module imports (R1 and R3) and method calls (R2 and R4).

Data Transmission. Attackers often manipulate malicious packages to enable data transmission, either by downloading payloads or sending sensitive information. In order to detect such behavior, we consider the import of network modules (D1) and the use of network-related method calls (D2) as suspicious indicators. Moreover, we identify string literals written in a URL format as an additional suspicious behavior (D3). By examining the package code, we search for strings that adhere to the format of a URL. This indicates the presence of potential data transmission activities, such as communication with external servers or services.

Encoding. Attackers often leverage encoding methods to obfuscate the malicious characteristics of their code, making it less noticeable and harder to detect. Therefore, we identify suspicious behavior related to the import of encoding modules (E1) and the presence of encoding calls within the code (E2). Besides, we also heuristically detect the use of base64 strings (E3) and the use of long strings (E4), which are known to be used in obfuscating or hiding malicious code [5].

Payload Execution. One of the goals of attackers is to execute the payload downloaded to the malicious package. This suspicious behavior encompasses the import of process modules (P1) and the use of process-related function calls (P2). For example, the call to `Popen` in the `subprocess`

```
1. <debug.py, 1, R1>   7. <pf.py,  8, D2>
2. <debug.py, 4, D1>   8. <pf.py, 13, R5>
3. <debug.py, 11, D2>  9. <pf.py, 21, R5>
4. <debug.py, 19, R2> 10. <pf.py, 23, R5>
5. <pf.py,  3, D1>    11. <pf.py, 25, R5>
6. <pf.py,  4, D3>
```

(a) Feature Instances

```
1. <debug.py,  1, R1>  7. <pf.py , 23, R5>
2. <pf.py ,  3, D1>    8. <pf.py,  25, R5>
3. <pf.py ,  4, D3>    9. <pf.py ,  13, R5>
4. <pf.py ,  8, D2>   10. <debug.py,  4, D1>
5. <debug.py , 19, R2> 11. <debug.py, 11, D2>
6. <pf.py,  21, R5>
```

(b) Behavior Sequence

Fig. 5. Extracted feature instances and generated behavior sequence for the package in Figure 3.

module in Python is considered as suspicious, as it allows the execution of arbitrary commands. The presence of such calls raises concerns about potential payload execution within the package. It could either be benign or malicious in our context. Further, we also identify suspicious behavior related to the use of bash scripts (P3). By analyzing the package code, we match patterns of commands, including *python<file>.py* and *wget https://url*. The detection of these patterns suggests the potential execution of commands or scripts that may facilitate payload execution. Moreover, we identify runtime code evaluation (P4) such as `eval` as a suspicious behavior as it may execute the downloaded payload.

Dynamic. Dynamic features are collected by executing a package, which is heavyweight. Only MalOSS [12] leverages these features, and we do not report the detailed list of dynamic features in Table 2. We do not consider dynamic features to strike a balance between effectiveness and efficiency.

The four abstraction dimensions, information reading, data transmission, encoding, and payload execution, are often used in combination to launch attacks. For example, attackers first read sensitive information and then send it through data transmission, or attackers first use data transmission to download payload, and then execute the payload. Encoding is leveraged to further hide the previous malicious behavior.

After introducing the feature set, we introduce how to extract these features. Generally, our feature extractor parses the **abstract syntax tree (AST)** of each source code file in a package to match patterns of features. Formally, the extracted feature instances are denoted as $\mathcal{F}$. Each feature instance f in $\mathcal{F}$ is denoted as a 3-tuple $\langle m, l, id \rangle$, where m and l, respectively, denote the method and the line number where the feature instance is extracted, and id denotes the ID of the feature.

Example 4.1. Figure 5(a) presents the extracted feature instances for the two source code files in Figure 3. For the ease of understanding, we use the source code file name instead of the method name in the extracted feature instances.

4.3 Behavior Sequence Generator

Our behavior sequence generator has three steps to organize the extracted feature instances into a behavior sequence based on their likelihood of execution and their sequential order in execution. The first step prioritizes the methods in a package according to the three triggering scenarios introduced in Section 3.1, which is used to determine the execution likelihood of a feature instance. The second step constructs the call graph of a package, which is used to determine the execution order of a feature instance. The last step generates the behavior sequence of a package by querying sub-call graphs in the order of prioritized methods and traversing them.

Prioritizing Methods. We first abstract the entire package into a collection of methods. The code at the global scope of each source code file is modeled into an implicit "main" method. Then, we identify and prioritize the methods based on their triggering scenarios. As discussed in Section 3.1,

there are three triggering scenarios, i.e., install-time, import-time and runtime execution. Overall, methods that are executed at install-time have high priority, followed by methods executed at import-time. Methods that are executed at runtime have low priority.

For methods executed at install-time, they are the implicit "main" methods of `setup.py` or script files specified in the *scripts* property of `package.json` by *preinstall*, *install* and *postinstall*. For methods executed at import-time, they consist of the implicit "main" methods in `__init__.py`, `index.js` and the imported module files. For methods executed at runtime, they encompass all publicly accessible methods, excluding private methods. In order to identify these methods, we need to exclude any method marked as private. In Python, a method is considered private when its name is prefixed with a double underscore ("__"). In JavaScript, private methods are indicated by a hash ("#") prefix before the method name [64].

When this step finishes, we obtain a prioritized method list, denoted as $\mathcal{M}$. Notice that the methods under the same triggering scenario are ranked in alphabet order by their names.

Constructing Call Graph. We employ static analysis techniques to construct the call graph of a package, which serves as the foundation for subsequent analysis. Specifically, we denote the call graph $\mathcal{G}$ as a 2-tuple $\langle \mathcal{M}, \mathcal{E} \rangle$, where $\mathcal{M}$ denotes the prioritized list of methods in the previous step, and $\mathcal{E}$ denotes the total set of calling edges. Each calling edge e in $\mathcal{E}$ is denoted as a 3-tuple $\langle m_a, l, m_b \rangle$, which means that there is a method call at line l in method m_a that calls method m_b.

Generating Behavior Sequence. We iterate each method r in $\mathcal{M}$ (i.e., in the order of execution likelihood) and query $\mathcal{G}$ to get a sub-call graph with its root being method r. We start from visiting r and traverse the sub-call graph in the order of execution to generate the behavior sequence.

During the sub-call graph traversal, when a method m is visited, we retrieve the feature instances extracted from m (denoted as $\mathcal{F}_m$) and obtain the calling edges starting from m (denoted as $\mathcal{E}_m$). Then, we enqueue $\mathcal{F}_m$ and $\mathcal{E}_m$ together into a queue in the order of their line numbers. In other words, given $f \in \mathcal{F}_m$ and $e \in \mathcal{E}_m$, we compare $e.l$ and $f.l$ to determine their execution order. Next, we dequeue elements from the queue. If the dequeued element is a feature instance f, we append f into the behavior sequence $\mathcal{S}$; and if the dequeued element is a calling edge e, we start to recursively visit the called method $e.m_b$. This makes Cerebro to preserve the sequential information through inter-procedural analysis, ensuring that the generated behavior sequence reflects the execution order.

Example 4.2. Figure 5(b) presents the generated behavior sequence when the implicit "main" method of `debug.py` is visited. Notice that this implicit "main" method is executed when `debug.py` is imported. We can see that the extracted feature instances in Figure 5(a) are organized according to their execution order in the two source code files in Figure 3.

4.4 Maliciousness Classifier

The maliciousness classifier leverages pre-trained language models to have a semantic representation of the behavior sequence from a package and then classify it as either malicious or benign. We adopt BERT [37], RoBERTa [50], and the encoder in T5 [67] as they are widely used. Specifically, we first transform $\mathcal{S}$ into a textual description $\mathcal{D}$ to facilitate the bi-lingual knowledge fusing as well as to ease the semantic understanding of behavior sequence. Then, by feeding textual descriptions of malicious and benign packages, we fine-tune the pre-trained language model into a binary classifier that fits for our task of malicious package detection in NPM and PyPI.

Transforming Behavior Sequence into Textual Description. We iterate each feature instance f in $\mathcal{S}$ and append the corresponding feature description (as shown in Table 2) into the textual description $\mathcal{D}$. When transforming the part of the behavior sequence that is generated from a method, we insert two short descriptions, namely *start entry <file_name>* and *end of entry*, to mark the beginning and

start entry superoptimzer/__init__.py, ... , end of entry, start entry superoptimzer/debug.py, import operating system module, import network module, use URL, use network module call, use operating system module call, use sensitive information, use sensitive information, use sensitive information, use sensitive information, import network module, use network module call, end of entry, start entry …

Fig. 6. Textual description of the behavior sequence generated for the package in Figure 3.

end of the feature instances originating from the same method. Consequently, we obtain a textual description for a package where the sequence of feature descriptions are concatenated through commas. As the pre-trained language model is exposed to massive textual data during pre-training, it is enabled to understand the semantics of certain security-related terms (e.g., "network," "URL," and "base64"). Hence, by using common words, the maliciousness classifier can benefit from the knowledge acquired by the pre-trained language model.

Example 4.3. Figure 6 depicts the textual description of the behavior sequence of the package in Figure 3. To differentiate the descriptions generated from different methods, we highlight them with different colors. The green part corresponds to the behavior executed in the implicit "main" method in `superoptimizer/__init__.py` at import-time. The blue part corresponds to the behavior executed in the implicit "main" method in `superoptimizer/debug.py` when `debug.py` is imported, which is the textual description of the behavior sequence in Figure 5(b).

Fine-Tuning Pre-Trained Language Models. We append a fully connected layer and a softmax layer to each pre-trained language model's architecture, and adopt the cross-entropy loss function to fine-tune the models into binary classifiers. We feed the model with textual descriptions of both malicious and benign packages from PyPI and NPM, enabling the classifier to learn bi-lingual knowledge. The position embedding module and self-attention mechanism in the pre-trained language model allow it to capture the sequential behavior knowledge.

4.5 Implementation

We have implemented Cerebro in 3.7K lines of Python code. To extract features, we employ tree-sitter [81] to transform Python and JavaScript code into an AST representation and identify suspicious behavior by matching the syntactic structures using AST queries provided by tree-sitter. To prioritize methods, we also use tree-sitter to parse each source code. To generate call graph, we leverage PyCG [69] for Python and Jelly [17, 52, 53, 55] for JavaScript. During the fine-tuning process, we employ the Adam optimizer with a learning rate of 1e-6 and a batch size of 1. The model is trained for three epochs to ensure optimal learning and performance.

5 Evaluation

We first present the evaluation setup and then report the evaluation results of **research questions (RQs)**.

5.1 Evaluation Setup

RQs. We design our evaluation to answer the following five RQs.

- *RQ1 Effectiveness Evaluation*: How is the effectiveness of Cerebro, compared with the state-of-the-art malicious package detection approaches?
- *RQ2 Efficiency Evaluation*: How is the performance overhead of Cerebro?
- *RQ3 Dataset Scale Evaluation*: How does the scale of the dataset affect the effectiveness of Cerebro?

Table 3. Statistics of the Dataset

Package Registry	Malicious			Benign		
	# Packages	Aver. Files #	Aver. KLOC #	# Packages	Aver. Files #	Aver. KLOC #
PyPI	887	5.6	0.987	2,398	75.3	19.492
NPM	1,788	4.4	1.642	4,993	214.3	9.256
Mixed	2,675	4.8	1.425	7,391	53.4	12.577

—*RQ4 Ablation Study*: How behavior sequence contributes to the effectiveness of Cerebro?
—*RQ5 Usefulness Evaluation*: How useful is Cerebro in real-world detection on PyPI and NPM?

Dataset Collection. We constructed the dataset as follows. For malicious packages, we collected malicious samples from three sources. We collected 438 malicious PyPI packages and 1,788 malicious NPM packages from Backstabber's Knife Collection [59]. We added 88 malicious PyPI packages from MalOSS's dataset [12]. We also sampled 361 packages out of a total of 5,874 packages from pypi_malregistry [89], with a confidence level of 95% and a margin of error of 5%. For benign packages, we collected 2,398 benign PyPI packages from the dataset of Vu et al. [85]. Following the procedure in prior work [57, 85], we selected the 5,000 most depended upon packages in the NPM registry as the dataset of benign NPM packages. However, we successfully downloaded 4,993 of them. In total, we curated a dataset of PyPI and NPM packages with 2,314 malicious packages and 7,391 benign packages, as listed in Table 3 where *Mixed* denotes the summation across PyPI and NPM. We also provide the average file count and average **thousand lines of code (KLOC)** for each package.

RQ Setup. For *RQ1*, we aim to compare the effectiveness of Cerebro with state-of-the-art approaches. Specifically, we selected Amalfi [70], SAP [43], and MPHunter [90] as the state-of-the-art learning-based approaches, and OSS Detect Backdoor [51] and Bandit4Mal [83] as the state-of-the-art rule-based approaches. We adopted the same configuration used in the original paper [43, 70, 90]. For OSS Detect Backdoor and Bandit4Mal, we set the threshold of three alerts to distinguish malicious and benign packages, which was observed as the optimal threshold by Vu et al. [85]. We did not compare with MalOSS [12] as it used dynamic features and thus failed to run on many packages. We also did not compare with Malware Checks [88] as the PyPI team removed it recently. We split the dataset into training and testing dataset by 9:1. We measured the precision and recall of all the approaches with 10-fold cross-validation on the testing datasets. We compared these approaches in three detection scenarios, i.e., mono-lingual scenario (i.e., train and test on the same ecosystem), cross-lingual scenario (i.e., train on one ecosystem and test on the other), and bi-lingual scenario (i.e., train on two ecosystems and test on one of the two ecosystems). We trained Cerebro with BERT [37], RoBERTa [50], and T5 [67] on the PyPI, NPM, and mixed training datasets, denoted by Cerebro$_{BERT}$, Cerebro$_{RoBERTa}$, and Cerebro$_{T5}$. We also trained Amalfi with decision tree (Amalfi$_{DT}$), naive bayes (Amalfi$_{NB}$), and SVM (Amalfi$_{SVM}$) on the same training datasets. We only evaluate SAP in both mono-lingual and bi-lingual scenarios, and MPHunter in the mono-lingual scenario for PyPI packages, as their do not support adaptation to other scenarios.

For *RQ2*, we measured the time overhead of Cerebro in offline training and online prediction. For *RQ3*, we evaluated the effectiveness of Cerebro across different dataset scales. We incrementally increased the proportion of our training dataset from 10% to 100% in 10% steps. We trained Cerebro and Amalfi$_{DT}$ in the bi-lingual scenario using these varying dataset proportions. The trained models were then tested using the same testing dataset across all scales. The average F1-Score over 10-fold cross-validation was used to measure overall effectiveness.

For *RQ4*, we created three ablated versions of Cerebro and compared their effectiveness with the original version of Cerebro using the same dataset in *RQ1*. Specifically, to evaluate the impact of sequential behavior, we created "Cerebro w/o Seq" by removing behavior sequence generator and feeding textual descriptions directly into maliciousness classifier. To evaluate the impact of textual description transformation, we created "Cerebro w/o Text" by removing textual description transformation in maliciousness classifier. To evaluate the impact of pre-trained language models, we created "Cerebro w/ DT" by removing behavior sequence generator and feeding features as a vector into the decision tree.

For *RQ5*, we ran Cerebro using the BERT model against the newly published packages in PyPI and NPM for over 9 months. This monitoring process started on 03 March 2023 for PyPI and 01 April 2023 for NPM, and ended at 30 October 2023. Cerebro analyzed 599,493 PyPI package versions and 324,145 NPM package versions. We manually confirmed the potentially malicious package versions flagged by Cerebro. Specifically, two authors manually assessed the maliciousness of the packages in the dimension of static characteristics, dynamic behaviors, and third-party detection tools. Two authors conducted a manual assessment to determine the maliciousness of the packages. This process covered comprehending the source code, inspecting the dynamic behaviors, and referencing detection results from third-party detection tools (e.g., virustotal [82]). The two authors are postgraduate and undergraduate students with security/malware background. The human inspection was conducted daily, costing 1 hour each day, over a period of 10 months. The average number of reviewed packages daily was 30. All packages confirmed by two authors as malicious were reported to the official PyPI and NPM teams, all of which were accepted by the two registries. Further, by adding the confirmed malicious and benign packages into the original training datasets (i.e., incremental learning), we retrained Cerebro using the BERT model to evaluate whether Cerebro could learn from new data to improve its effectiveness. We also analyzed the malicious intentions and triggering scenarios of the new malicious packages.

$$\begin{aligned} \text{Pre.} &= \frac{\mid MP_{tool} \cap MP_{test} \mid}{\mid MP_{tool} \mid} \\ \text{Rec.} &= \frac{\mid MP_{tool} \cap MP_{test} \mid}{\mid MP_{test} \mid} \\ \text{F1-Score} &= \frac{2 \times \text{Pre.} \times \text{Rec.}}{\text{Pre.} + \text{Rec.}} \end{aligned} \tag{1}$$

Evaluation Metric. The precision (Pre.), recall (Rec.) used in *RQ1*, *RQ4*, *RQ5,* and F1-Score used in *RQ3* are defined in Equation (1). Specifically, MP_{tool} denotes the set of malicious packages predicted by tools (e.g., Cerebro), and MP_{test} denotes the set of true malicious packages in the testing dataset. Precision is calculated by the proportion of true positives (i.e., $MP_{tool} \cap MP_{test}$) in the predicted malicious packages (i.e., MP_{tool}), and recall is calculated by the proportion of true positives (i.e., $MP_{tool} \cap MP_{test}$) in the ground truth (i.e., MP_{test}).

5.2 Effectiveness Evaluation (RQ1)

Mono-Lingual Scenario. The result of learning-based approaches in mono-lingual scenario is reported in Table 4, and the result of rule-based approaches is reported in Table 5. Overall, Cerebro$_{RoBERTa}$ outperforms all the other approaches. In PyPI, Cerebro$_{RoBERTa}$ achieves a precision of 96.0% and a recall of 91.7%. It achieves the best result, with slight advantages compared with Cerebro$_{BERT}$ and Cerebro$_{T5}$. It outperforms the state-of-the-art SAP by 12.1% in precision, and MPHunter by 6.3% in recall. In NPM, Cerebro$_{RoBERTa}$ achieves a precision of 98.5% and a recall of 92.9%. It outperforms Cerebro$_{T5}$ with a 1.9% higher recall and a 0.4% lower precision, and Cerebro$_{BERT}$ with a 0.3% higher precision and 1.1% higher recall. It outperforms the state-of-the-art Amalfi$_{DT}$

Table 4. Evaluation Result in Mono-Lingual Scenario for Learning-Based Approaches

Train	Test	CEREBRO$_{BERT}$		CEREBRO$_{RoBERTa}$		CEREBRO$_{T5}$		AMALFI$_{DT}$		AMALFI$_{NB}$		AMALFI$_{SVM}$		SAP		MPHUNTER	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
PyPI	PyPI	95.8%	89.4%	96.0%	91.7%	93.4%	70.9%	81.9%	83.9%	59.5%	7.5%	30.3%	30.9%	83.9%	61.5%	21.8%	85.4%
NPM	NPM	98.2%	91.8%	98.5%	92.9%	98.9%	91.0%	92.7%	86.0%	92.1%	77.1%	10.2%	26.7%	90.5%	35.4%	–	–

Table 5. Evaluation Result in Mono-Lingual Scenario for Rule-Based Approaches

Test	OSS Detect Backdoor		Bandit4Mal	
	Pre.	Rec.	Pre.	Rec.
PyPI	19.9%	60.4%	24.9%	74.4%
NPM	41.1%	85.1%	–	–

Table 6. Evaluation Result in Cross-Lingual Scenario

Train	Test	CEREBRO$_{BERT}$		CEREBRO$_{RoBERTa}$		CEREBRO$_{T5}$		AMALFI$_{DT}$		AMALFI$_{NB}$		AMALFI$_{SVM}$	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
NPM	PyPI	72.0%	49.1%	75.1%	44.1%	65.2%	63.4%	72.0%	18.0%	42.0%	31.5%	38.4%	76.0%
PyPI	NPM	41.9%	93.0%	46.5%	92.7%	37.5%	90.9%	14.4%	4.7%	48.8%	5.2%	28.8%	94.6%

by 5.8% in precision and 6.9% in recall. The two rule-based approaches have low precision and recall.

Cross-Lingual Scenario. The result of cross-lingual scenario is presented in Table 6. When the model is trained on NPM packages and tested on PyPI packages, CEREBRO$_{RoBERTa}$ obtains a highest precision of 75.1%. However, CEREBRO$_{T5}$ achieves a more balanced performance with a precision of 65.2% and a recall of 63.4%. Although AMALFI$_{SVM}$ has better recall, CEREBRO$_{T5}$ significantly surpasses it in precision, resulting in an overall F1-Score advantage of 13.3%. When the model is trained on PyPI packages and tested on NPM packages, CEREBRO$_{RoBERTa}$ obtains a highest precision of 46.5% and a recall of 92.7%. Although AMALFI$_{SVM}$ has better recall, CEREBRO$_{RoBERTa}$ significantly surpasses it in precision, resulting in a F1-Score advantage of 17.7%. This result potentially owes to our well-abstracted feature set. Notice that CEREBRO in the cross-lingual scenario achieves a lower precision and recall than in the mono-lingual scenario, which indicates that some malicious behaviors might be not common, and thus bi-lingual knowledge fusing is necessary.

Bi-Lingual Scenario. The result of bi-lingual scenario is shown in Table 7. Overall, CEREBRO$_{RoBERTa}$ outperforms all the other approaches in PyPI and NPM packages. For testing PyPI packages using the model trained on mixed packages, CEREBRO$_{BERT}$ obtains a precision of 95.0% and a recall of 92.0%, and CEREBRO$_{RoBERTa}$ obtains a precision of 96.1% and a recall of 90.9%. They achieve very similar results, with CEREBRO$_{RoBERTa}$ outperforming CEREBRO$_{BERT}$ by 1.1% in precision, while CEREBRO$_{BERT}$ surpasses CEREBRO$_{RoBERTa}$ by the same margin in recall. Comparing with the state-of-the-art, CEREBRO$_{RoBERTa}$ outperforms AMALFI$_{DT}$ by 12.9% in precision and 9.8% in recall. For testing NPM packages using the model trained on mixed packages, CEREBRO$_{RoBERTa}$ achieves a precision of 98.9% and a recall of 93.9%. It outperforms CEREBRO$_{BERT}$ by 0.2% in precision and 0.9% in recall and surpasses CEREBRO$_{T5}$ by 1.9% in recall. Comparing with the state-of-the-art, CEREBRO$_{RoBERTa}$ outperforms AMALFI$_{DT}$ by 6.8% in precision 8.1% in recall. Although AMALFI$_{SVM}$ outperforms CEREBRO in recall by 0.7%, it suffers from a very low precision of 28.8%.

Comparing Bi-Lingual with Mono-Lingual. Comparing the effectiveness of CEREBRO in bi-lingual and mono-lingual scenarios, we observe that CEREBRO$_{BERT}$, CEREBRO$_{RoBERTa}$, and CEREBRO$_{T5}$

Table 7. Evaluation Result in Bi-Lingual Scenario

Train	Test	Cerebro$_{BERT}$		Cerebro$_{RoBERTa}$		Cerebro$_{T5}$		Amalfi$_{DT}$		Amalfi$_{NB}$		Amalfi$_{SVM}$		SAP	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
Mixed	PyPI	95.0%	92.0%	96.1%	90.9%	94.6%	69.6%	83.2%	81.1%	42.0%	35.6%	45.1%	37.2%	80.2%	60.2%
	NPM	98.7%	93.0%	98.9%	93.9%	98.9%	92.0%	92.1%	85.8%	93.9%	76.6%	28.8%	94.6%	90.4%	39.5%

Table 8. Efficiency Evaluation Results

Training Data	Training Time (hours)	Average Prediction Time (seconds)			
		Feature Extractor	Behavior Sequence Generator	Maliciousness Classifier	Total
PyPI	14.1	3.737	7.807	0.008	11.552
NPM	25.2	2.412	7.611	0.007	10.030
Mixed	39.3	2.856	7.664	0.007	10.527

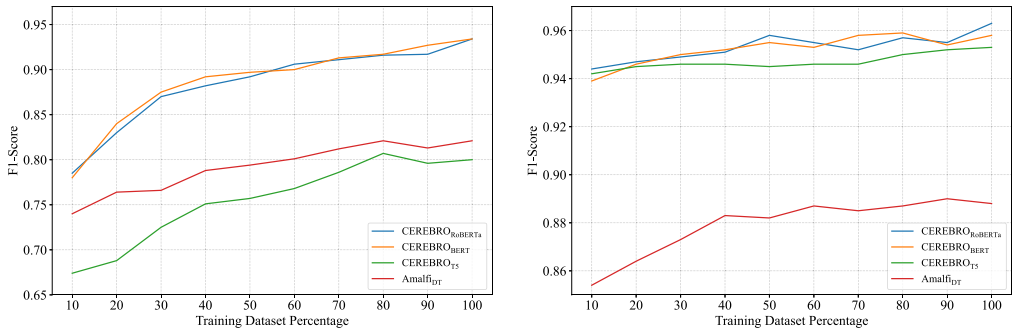
trained on mixed packages outperform their mono-lingual counterparts. For detecting malicious PyPI packages, the bi-lingual model of Cerebro achieves an average precision increase of 0.2% and an average recall increase of 0.2%. For NPM packages, the bi-lingual model achieves an average precision increase of 0.3% and an average recall increase of 1.1%. These results indicate that our bi-lingual knowledge fusion is useful.

When comparing the performance of Cerebro$_{BERT}$ and Cerebro$_{RoBERTa}$, we observe that Cerebro$_{RoBERTa}$ generally outperforms Cerebro$_{BERT}$ in most scenarios. This advantage can be attributed to RoBERTa's use of more advanced training strategies, including dynamic masking, larger batch sizes, and an increased number of training steps, which enable the model to learn more efficiently. However, in cross-lingual and bi-lingual tasks, Cerebro$_{BERT}$ achieves higher or comparable recall compared to Cerebro$_{RoBERTa}$. Further analysis reveals that for malicious packages detected by Cerebro$_{BERT}$ but missed by Cerebro$_{RoBERTa}$, the sequences tend to be longer. This suggests that Cerebro$_{RoBERTa}$ may be less effective at capturing long-range dependencies within longer sequences, which are essential for recognizing complex malicious patterns. A potential explanation for this discrepancy lies in the pre-training tasks of the two models: RoBERTa does not include the **next sentence prediction (NSP)** task during its training, while BERT does. The inclusion of NSP in BERT's training may enhance its ability to comprehend long-context relationships, making it better at detecting certain malicious packages that RoBERTa misses.

Summary. Cerebro$_{RoBERTa}$ outperforms the state-of-the-art by averagely 10.0% in precision and 7.4% in recall in the mono-lingual scenario, and averagely 9.9% in precision and 8.9% in recall in the bi-lingual scenario. Cerebro learns bi-lingual knowledge from two package registries, which contributes to an average increase of 0.3% in precision and 0.7% in recall.

5.3 Efficiency Evaluation (RQ2)

We measure the time overhead of Cerebro for training and prediction, including all components described in Section 4. Cerebro is trained on a machine with an Intel Xeon Silver 4314 CPU at 2.40 GHz, 128G of RAM and an NVIDIA GeForce RTX 3090. The same machine is employed for predictions. Table 8 presents the efficiency results. To train Cerebro with packages from a single package registry, it takes around 14.1 hours for PyPI packages and 25.2 hours for NPM packages. To equip Cerebro with bi-lingual knowledge using the mixed packages, the training time is proportional to the scale of the dataset, which amounts to 39.3 hours. Moreover, the average prediction time for a package is similar for BERT, RoBERTa, or T5 trained on PyPI, NPM, and mixed packages, which is respectively 11.552, 10.030, and 10.527 seconds.



(a) PyPI Packages as the Testing Dataset (b) NPM Packages as the Testing Dataset

Fig. 7. Effectiveness of CEREBRO$_{BERT}$, CEREBRO$_{RoBERTa}$, CEREBRO$_{T5}$, and AMALFI$_{DT}$ in different scales of dataset.

Summary. CEREBRO takes less than 2 days to train the model; and CEREBRO takes an average of 10.5 seconds to predict whether a package is malicious, which is acceptable.

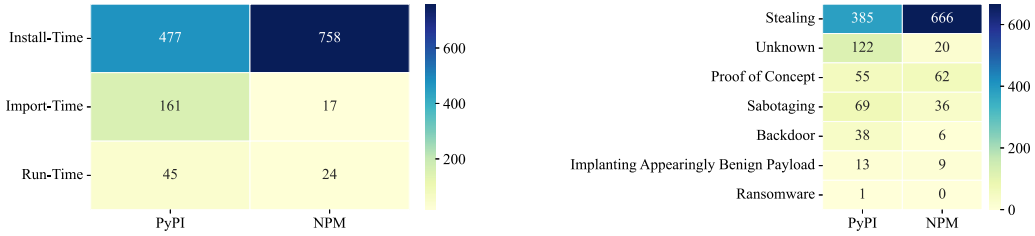
5.4 Dataset Scale Evaluation (RQ3)

Figure 7 presents the results of our dataset scale evaluation. Overall, CEREBRO$_{BERT}$ and CEREBRO$_{RoBERTa}$ consistently outperform the state-of-the-art by a significant margin in both PyPI and NPM packages, even with limited dataset scales (i.e., 10%, 20%). In Figure 7(a), CEREBRO$_{BERT}$ and CEREBRO$_{RoBERTa}$ exhibit similar F1-Scores as the dataset scale changes. CEREBRO$_{RoBERTa}$ outperforms AMALFI$_{DT}$ by an F1-Score of 0.045 at the 10% scale. As the dataset scale increases, CEREBRO$_{RoBERTa}$ further widens the gap, surpassing AMALFI$_{DT}$ with an F1-Score of 0.113 at the 100% scale. In Figure 7(b), CEREBRO$_{BERT}$, CEREBRO$_{RoBERTa}$, and CEREBRO$_{T5}$ exhibit similar F1-Scores as the dataset scale changes. CEREBRO$_{RoBERTa}$ outperforms AMALFI$_{DT}$ by an F1-Score of 0.09 at the 10% scale. As the dataset scale increases, CEREBRO$_{RoBERTa}$ further keeps the gap, achieving an average F1-Score difference of 0.072 across remaining dataset scales. We also observe that as the training dataset increases, the F1-Score improvement for NPM packages is relatively smaller compared to PyPI packages. This is primarily due to the differing levels of package diversity between the two ecosystems. As shown in Figure 8(b), a larger proportion of NPM packages focus on information stealing. Consequently, models trained on NPM packages quickly learn and generalize malicious patterns from smaller datasets than those for PyPI, resulting in better performance at lower data percentages and smaller incremental gains as the dataset scale increases.

Summary. The effectiveness of our approach maintains an advantage over state-of-the-art across different dataset scales. CEREBRO$_{RoBERTa}$ achieves an F1-Score of 0.785 in PyPI and 0.944 in NPM even when the dataset scale is reduced by 90%, indicating the robustness of CEREBRO in situations with scarce data availability.

5.5 Ablation Study (RQ4)

We train the three ablated versions of CEREBRO in the bi-lingual scenario. As shown in Table 9, the ablated version is less effective than the original version. Specifically, the average precision and recall for CEREBRO$_{BERT}$, CEREBRO$_{RoBERTa}$, and CEREBRO$_{T5}$ decrease 1.6% and 0.6%, increase 1.6% and decrease 3.7%, decrease 1.4% and 1.9% for CEREBRO w/o Seq in mono-lingual, cross-lingual, and bi-lingual scenarios, respectively. Similarly, the average precision and recall decrease 1.4% and 1.2%, decrease 2.0% and 33.0%, decrease 0.5% and 0.7% for CEREBRO w/o Text in mono-lingual, cross-lingual, and bi-lingual scenarios, respectively. Further, the average precision and recall decrease



(a) Triggering Scenarios of Malicious Packages

(b) Malicious Intentions of Malicious Packages

Fig. 8. Triggering scenarios and malicious intentions of new malicious packages.

Table 9. Evaluation Result on Ablated Versions of Cerebro

Train	Test	Cerebro w/o Seq						Cerebro w/o Text						Cerebro w/ DT	
		BERT		RoBERTa		T5		BERT		RoBERTa		T5		Pre.	Rec.
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.		
PyPI	PyPI	91.3%	87.1%	92.1%	86.2%	92.1%	76.1%	92.6%	85.0%	90.6%	89.6%	94.1%	76.8%	91.4%	89.4%
		↓ 4.5%	**↓ 2.3%**	**↓ 3.9%**	**↓ 5.5%**	**↓ 1.3%**	**↑ 5.2%**	**↓ 3.2%**	**↓ 4.4%**	**↓ 5.4%**	**↓ 2.1%**	**↑ 0.7%**	**↑ 5.9%**	**↓ 4.4%**	**↓ 0.0%**
NPM	NPM	98.0%	92.2%	98.9%	92.1%	98.8%	90.4%	98.1%	90.4%	97.7%	90.4%	99.1%	88.3%	96.6%	92.2%
		↓ 0.2%	**↑ 0.4%**	**↑ 0.4%**	**↓ 0.8%**	**↓ 0.1%**	**↓ 0.6%**	**↓ 0.1%**	**↓ 1.4%**	**↓ 0.8%**	**↓ 2.5%**	**↑ 0.2%**	**↓ 2.7%**	**↓ 1.6%**	**↑ 0.4%**
NPM	PyPI	77.3%	36.0%	76.7%	42.1%	64.7%	67.6%	93.7%	18.5%	64.2%	2.0%	88.0%	8.4%	49.5%	11.0%
		↑ 5.3%	**↓ 13.1%**	**↑ 1.6%**	**↓ 2.0%**	**↓ 0.5%**	**↑ 4.2%**	**↑ 21.7%**	**↓ 30.6%**	**↓ 10.9%**	**↓ 42.1%**	**↑ 22.8%**	**↓ 55.0%**	**↓ 22.5%**	**↓ 38.1%**
PyPI	NPM	49.2%	92.2%	38.3%	87.2%	41.3%	86.0%	41.8%	94.0%	28.0%	98.9%	10.6%	13.6%	10.2%	8.4%
		↑ 7.3%	**↓ 0.8%**	**↓ 8.2%**	**↓ 5.5%**	**↑ 3.8%**	**↓ 4.9%**	**↓ 0.1%**	**↑ 1.0%**	**↓ 18.5%**	**↑ 6.2%**	**↓ 26.9%**	**↓ 77.3%**	**↓ 31.7%**	**↓ 84.6%**
Mixed	PyPI	92.3%	86.2%	93.4%	88.3%	92.6%	71.3%	95.5%	86.0%	94.7%	87.4%	92.9%	82.0%	90.5%	86.6%
		↓ 2.7%	**↓ 5.8%**	**↓ 2.7%**	**↓ 2.6%**	**↓ 2.0%**	**↑ 1.7%**	**↑ 0.5%**	**↓ 6.0%**	**↓ 1.4%**	**↓ 3.5%**	**↓ 1.7%**	**↑ 12.4%**	**↓ 4.5%**	**↓ 5.4%**
	NPM	98.2%	91.8%	98.6%	91.7%	98.9%	90.9%	98.7%	91.1%	98.3%	91.1%	98.8%	89.5%	93.9%	92.0%
		↓ 0.5%	**↓ 1.2%**	**↓ 0.3%**	**↓ 2.2%**	**↓ 0.0%**	**↓ 1.1%**	**↓ 0.0%**	**↓ 1.9%**	**↓ 0.6%**	**↓ 2.8%**	**↓ 0.1%**	**↓ 2.5%**	**↓ 4.8%**	**↓ 1.0%**

The bold numbers indicate the decreases or increases compared to the non-ablated versions.

3.0% and 0.2%, 27.1% and 61.4%, 4.7% and 3.2% for Cerebro w/ DT in mono-lingual, cross-lingual, and bi-lingual scenarios, respectively.

Summary. Generating behavior sequence, transforming behavior sequence into textual description, and feeding textual description into a classifier fine-tuned from a pre-trained language model are all effective in detecting malicious packages. Averagely, the ablated versions of Cerebro decrease 4.5% in precision and 11.8% in recall.

5.6 Real-World Usefulness Evaluation (RQ5)

To evaluate the practical usefulness of Cerebro in analyzing newly published package versions in real-world registries, we design a monitoring system that first monitors and crawls newly published package versions from PyPI and NPM and then runs Cerebro against the crawled package versions. Our monitoring system has lasted over 8 months for PyPI and 7 months for NPM.

Overall Usefulness Result. In total, Cerebro scans 923,638 newly published package versions, and flags 3,746 PyPI package versions and 2,230 NPM package versions as potentially malicious, as reported in Table 10. We first manually validate each flagged package version and confirm 683 PyPI package versions and 799 NPM package versions as true positives. Then, following responsible disclosure process, we promptly report these package versions to administrators of the official PyPI and NPM teams. At the time of reporting, 451 PyPI package versions and 324 NPM package versions have already been removed. The PyPI and NPM teams confirm the remaining 232 PyPI package versions and 475 NPM package versions as malicious and remove all of them. Thus, we receive 232 and 457 thank letters from PyPI and NPM.

False-Positive Analysis. As shown in Table 10, the false-positive rate is 81.5% for PyPI and 64.2% for NPM, which is actually high. It is worth mentioning that it takes one of our authors around

Table 10. Result of Real-World Malicious Package Detection

Package Registry	Time Period	Newly Published Package Versions	Flagged by Cerebro	False Positives	True Positives	Thank Letters	Removed Before Report
PyPI	March	104,512	750	521	229	173	56
	April	101,415	661	560	101	0	101
	May	78,528	422	324	98	30	68
	June	102,088	768	595	173	0	173
	July	101,600	720	670	50	10	40
	August	56,561	298	283	15	14	1
	September	25,193	101	88	13	4	9
	October	29,596	16	12	4	1	3
	Total	599,493	3,746	3,053	683	232	451
NPM	April	130,697	325	178	147	147	0
	May	39,094	145	88	57	35	22
	June	28,764	256	145	111	0	111
	July	34,626	244	155	89	44	45
	August	33,013	514	314	200	131	69
	September	27,350	374	254	120	91	29
	October	30,601	372	297	75	27	48
	Total	324,145	2,230	1,431	799	475	324

1 hour to manually analyze and confirm the flagged package versions every day, which is still acceptable. After analyzing all the false positives, we identify four major types of benign packages that are flagged as malicious, as their behavior sequences are similar to malicious ones.

First, some benign packages encapsulate RESTful APIs, which enable data transmission to remote servers. Their behavior sequences consist of features about *Information Reading* and *Data Transmission* (e.g., R1, D1, D2, and D3). Second, some benign packages encapsulate CLI commands into Python/JavaScript methods where their behavior sequences resemble those malicious packages in using features about *Payload Execution*. Third, some benign packages behave similarly to Trojans, but their payloads serve benign purposes. Normally, their behavior sequences resemble Trojans by starting with D1 and D3, then undertaking intermediate actions such as R1 and R4, and ending with P2 and P3. Last, some benign packages consist of a single meta utility module that incorporates features ranging from *Information Reading*, *Data Transmission*, *Encoding* to *Payload Execution*. It causes Cerebro to recognize parts of behavior sequences as similar to those of malicious packages.

False-Negative Analysis. To measure false negatives of Cerebro, we reached out to package registries. However, PyPI replied that they did not maintain a malicious package list and NPM refused to share the list. Therefore, we try to use state-of-the-art detectors to identify true positives. Specifically, we employ Amalfi [70] to identify potentially malicious packages during the same monitoring time period. It flagged 41,434 PyPI package versions and 1,930 NPM package versions as potentially malicious. Then, we undertake a manual inspection process with a sampling rate of 95% confidence level and 5% error margin, which results in 381 PyPI packages and 164 NPM packages. After manual validation, we identify 9 malicious PyPI packages and 45 malicious NPM packages. Remarkably, only two PyPI packages and five NPM packages went undetected by Cerebro. It indicates that the false-negative rate of Cerebro is relatively low in real-world detection.

Incremental Learning Result. As we are continuously confirming and accumulating true positives and false positives of malicious packages based on our monitoring system, one potential way to reduce false positives is to adopt incremental learning on such true positives and false positives. To this end, we add the true positives and false positives confirmed during our monitoring from March to June as two incremental datasets for retraining. In particular, we add 750 packages from March,

Table 11. Result of Incremental Learning

Added Train (#)	Time Period	New Test (#)	Pre.
None	–		7.2%
PyPI (750)	March		26.5%
Mixed (1,736)	March–April	PyPI (1,135)	36.5%
Mixed (2,303)	March–May		39.0%
Mixed (3,327)	March–June		58.2%
None	–		32.2%
PyPI (750)	March		76.3%
Mixed (1,736)	March–April	NPM (1,504)	80.3%
Mixed (2,303)	March–May		80.2%
Mixed (3,327)	March–June		80.7%

1,736 packages from March to April, 2,303 packages from March to May, and 3,327 packages from March to June, as reported in the third to sixth rows, and 8 to 11 rows under the column of *Added Train* in Table 11. Note that the added training dataset for March contains only PyPI packages because monitoring NPM packages starts from April. Then, we retrain four models of Cerebro by augmenting the original mixed training dataset with the new mixed PyPI and NPM packages. Finally, we select true positives and false positives confirmed during our monitoring from July to October as the new testing dataset. In particular, there are 1,135 PyPI packages and 1,504 NPM packages as reported in the third column in Table 11.

Overall, by incorporating new malicious and benign packages into the training dataset, the precision of Cerebro improves significantly. Specifically, when Cerebro is retrained with new mixed packages, the precision of detecting malicious PyPI packages reaches 58.2%, while the precision of detecting malicious NPM packages reaches 80.7%, achieving a precision increase of 51.0% for PyPI packages and 48.5% for NPM packages in total.

Malicious Package Characteristics. For each of the 683 PyPI and 799 NPM malicious package versions which are previously unknown we manually look into their malicious behavior to understand their characteristics. On the one hand, we determine their triggering scenarios, and report the result in Figure 8(a). We can observe that 477 (69.8%) of the malicious PyPI package versions and 758 (94.9%) of the malicious NPM package versions trigger their malicious behavior at install-time. This is reasonable because install-time execution can achieve the highest probability of successfully carrying out attacks. Moreover, a small number of malicious PyPI package versions opt to trigger malicious behavior other than install-time, accounting for 161 (23.6%) at import-time and 45 (6.6%) at runtime. Noticeably, there are only 17 (2.1%) of the malicious NPM package versions that adopt import-time triggering, and 24 (3.0%) malicious NPM package versions adopt runtime triggering.

On the other hand, we explore the intentions of malicious package versions by executing them in a sandbox, and present the result in Figure 8(b). Generally, the malicious intentions include stealing (e.g., personal data or credentials), sabotaging (e.g., shutting down firewalls), proof of concept (which is harmless but to demonstrate something malicious can be done), backdoor (i.e., enabling remote-controlled operations for an attack to exploit an infected machine), implanting appearingly benign payload (e.g., the benign payload might change to be malicious via morphing Trojan), and ransomware. Besides, we fail to execute the payload for 142 of the malicious package versions due to non-existent URLs or runtime exceptions, and thus we label their malicious intentions as unknown.

Summary. Cerebro detects 683 and 799 previously unknown malicious PyPI and NPM package versions. All of them have been removed by the official PyPI and NPM teams, and we also receive 707 thank letters from them. Further, by incorporating new malicious and benign packages, Cerebro

learns new knowledge to improve its precision. Besides, most (83.3%) of these new malicious package versions execute the malicious behavior at install-time; and most (70.9%) of them attempt to steal sensitive information.

5.7 Threats and Limitations

Threats. First, as the campaign against malware continues, new strains of malicious packages emerge daily. This implies that the original dataset of malicious packages used in our evaluation may not encompass the full spectrum of new malicious packages, hindering Cerebro's effectiveness. However, we have demonstrated the effectiveness of Cerebro in learning from newly published packages and adapting to detect emerging threats. Second, while we thoroughly examine the potentially malicious package versions identified by Cerebro, we acknowledge that the vast workload makes it impractical to validate every potentially benign package version. Therefore, we only systematically report the result of precision but not the result of recall. Third, Cerebro can generate false positives. To address this, we conduct manual validation on identified package versions before reporting them to PyPI and NPM teams. While this process can also introduce false positives, the fact that the reported packages are subsequently removed indicates the validity of our validation. Fourth, as pre-trained language models continue to advance rapidly, we expect to see new and more powerful models. Nevertheless, the choice of language models is orthogonal to our approach. We plan to test the effectiveness of additional language models on this task. Fifth, the varying behavior sequences between malicious and benign packages may reduce the accuracy of Cerebro in cross-lingual scenarios. Nevertheless, the cross-lingual setting represents an extreme case in real-world where there are no malicious samples within the same ecosystem.

Limitations. First, our method is designed for PyPI and NPM packages. However, our method can be easily generalized into other ecosystems by implementing the feature extractor and behavior sequence generator for a new ecosystem. The feature set is language-agnostic. By leveraging the existing AST parser and call graph generator of the new ecosystem, we believe that the integration can be straightforward. Second, large packages may have a large size of the behavior sequence which can exceed the 512 token limit of BERT. However, based on our analysis of packages in Table 10, we identify a relatively modest proportion (13.6%) of packages that exceed the limit. We plan to split the behavior sequence into multiple segments to circumvent the limit while maintaining their sequential integrity. Third, the proposed approach encounters higher false positives in real-world setting. We foresee several challenges and opportunities that could help reduce false positive rates. On the one hand, we can leverage ensemble methods. Utilizing a combination of different detection tools could be advantageous. An ensemble approach, where decisions are based on the consensus among multiple tools, might improve accuracy and reduce false-positive rates. On the other hand, we can utilize dynamic analysis during our detection. Incorporating dynamic analysis into the detection process could provide deeper insights into the behavior of packages by capturing detailed runtime behaviors, potentially reducing false positives by confirming suspicious activities through actual execution rather than static analysis. Nonetheless, with a large number of new packages being released daily, it remains an open question how to leverage the benefits of dynamic analysis while mitigating its computational and time costs.

6 Related Work

Malicious Package Detection. Recently, there has been an increasing number of OSS supply chain attacks that inject malicious code into packages, with package registries such as NPM and PyPI being the largest targets [44, 54]. Ohm et al. [59] were the first to systematically investigate malicious packages. They collected 174 real-world malicious packages from NPM, PyPI, and RubyGems. Using

this dataset, they manually analyzed how malicious behavior was injected (e.g., via typosquatting) and triggered (e.g., on installation), what the objective (e.g., data exfiltration) of malicious behavior was, and whether obfuscation was employed. Similarly, Wenbo et al. [89] conducted an empirical study to understand the characteristics of the malicious code lifecycle in the PyPI ecosystem. Various approaches have also been proposed to detect malicious packages, mostly in NPM and PyPI.

In the NPM ecosystem, Zahan et al. [94] identified six signals of security weakness in NPM supply chain and proposed a rule-based malicious package detector that searches for keywords in install scripts. Garrett et al. [21] selected features about whether libraries that access the network, file system, and OS processes are used, whether code is evaluated at runtime, and whether new files, new dependencies, and new hookup script entries are present, and adopted clustering to build a benign behavior model which is used to detect malicious NPM packages. Differently, Ohm et al. [58] adopted AST-level clustering to establish a malicious behavior model. Instead of using unsupervised methods, Fass et al. [15, 16] built a random forest classifier based on the frequency of specific patterns extracted from AST, control flow graph, and program dependency graph of benign and malicious JavaScript samples. This approach is specifically designed for obfuscated JavaScript programs and thus could be ineffective for non-obfuscated ones. Ohm et al. [57] extended the feature set of Garrett et al.'s work [21] by including features about package metadata and obfuscation. Similarly, Sejfia and Schäfer [70] proposed Amalfi, which uses an extended feature set from Garrett et al.'s work [21] to train classifiers. These approaches capture malicious behavior as discrete features, hindering their accuracy in detecting malicious packages. Differently, we model malicious behavior in a sequential manner, which can model the maliciousness more precisely. Notice that Ferreira et al. [18] proposed a lightweight permission system to protect applications from malicious package updates at runtime. Recently, Huang et al. [33] propose SpiderScan to identify malicious NPM packages based on graph-based behavior modeling and matching.

In the PyPI ecosystem, Vu et al. [87] proposed a rule-based approach to detect malicious PyPI packages that are spread by typosquatting and combosquatting attacks. The rule checks whether the package has the name similar to a Python standard library or a package with known source code repository. Besides, there are several other rule-based detectors, e.g., Bandit4Mal [83] and Malware Checks [88]. Vu et al. [85] conducted a comprehensive evaluation of these detectors. Instead of relying on rules, Vu et al. [84, 86] identified malicious packages based on discrepancies between the source code repository and distributed artifact of a Python package. Liang et al. [47] used both package metadata features and code features to learn an isolated forest model for detecting malicious packages. Recently, Wentao et al. [90] utilized clustering techniques to group the PyPI installation scripts so that outliers can be identified. Sun et al. [76] propose to integrate deep code behavior features with metadata features to detect malicious PyPI packages. These approaches share the same limitation with those detectors in NPM because they also model malicious behavior as discrete features.

Moreover, some approaches support different ecosystems. OSS Detect Backdoor [51] is a rule-based detector that support 15 ecosystems. Taylor et al. [80] leveraged package name similarity and package popularity to detect malicious packages caused by typosquatting in NPM, PyPI, and RubyGems. Ladisa et al. [43] utilized language-independent feature (e.g., number of URLs, entropy of strings in code files) to detect malicious packages in NPM and PyPI. However, these approaches are not based on high-level semantics of source code, which can produce high false positives. Different from the simple rule in Taylor et al. and Ladisa et al., Duan et al. [12] derived five metadata analysis rules, four static analysis rules, and four dynamic analysis rules to detect malicious packages in NPM, PyPI, and RubyGems. However, it is heavyweight as it relies on program analysis.

Squatting attacks are also common in other domains, e.g., domain name system [1, 38, 40, 78, 79], container registry [49], and mobile app [31]. Malware detection has been widely studied [89, 93]. OSS opens up new opportunities for malware detection due to the availability of source code.

Security Threats in OSS Supply Chain. Apart from malicious packages, there exist various security threats in OSS supply chain. Several studies [3, 7, 29, 36, 94] have been conducted to investigate security threats in package registries such as NPM and PyPI. Ladisa et al. [42] reported a taxonomy of attacks on all OSS supply chain stages from code contributions to package distribution. They also assessed the safeguards against OSS supply chain attacks. Koishybayev et al. [41] and Gu et al. [28] characterized security threats in continuous integration workflows that produce OSS.

Enck and Williams [13] summarized five challenges in OSS supply chain security, with the top one being updating vulnerable dependencies. Specifically, the vulnerability impact analysis along the supply chain has been widely explored in various ecosystems, e.g., NPM [48, 95, 96], Maven [25, 32, 63], PyPI [2, 63, 68], and RubyGems [63, 95]. To cope with vulnerable dependencies, various software composition analysis tools have been proposed [32, 48, 62, 92]. Besides, several advances have been made on detect maliciousness in different stages of software development. Goyal et al. [27] and Gonzalez et al. [26] proposed to detect malicious commits on GitHub. Cao and Dolan-Gavitt [6] tried to identify malware in GitHub forks. Lamb and Zacchiroli [46] and Wheeler [91] attempted to avoid the injection of maliciousness in compilation by ensuring reproducible builds.

7 Conclusion and Future Work

We have proposed and implemented Cerebro to detect malicious packages in NPM and PyPI. Cerebro leverages a comprehensive feature set that models high-level abstraction of malicious behavior sequence, enabling bi-lingual knowledge fusing. Our extensive evaluation results have demonstrated the effectiveness, efficiency, and practical usefulness of Cerebro in detecting malicious packages. In the future, we plan to enhance the multi-lingual capability of Cerebro by incorporating support for both interpreted and compiled languages. The challenge is to design feature extractors for new languages and improve the generalizability of our feature set. We also plan to equip Cerebro with the capability to pinpoint the specific malicious sequence from the whole behavior sequence, further aiding administrators to go through the manual confirmation promptly.

Data Availability

The data of our evaluation are available at https://doi.org/10.5281/zenodo.8277447.

References

[1] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months' worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium*, 1–13.

[2] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. 2023. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering* 28, 3 (2023), 59.

[3] Aadesh Bagmar, Josiah Wedgwood, Dave Levin, and Jim Purtilo. 2021. I know what you imported last summer: A study of security threats in the python ecosystem. arXiv:2102.06301. Retrieved from https://arxiv.org/abs/2102.06301

[4] Alex Birsan. 2021. Dependency confusion: How I hacked into Apple, Microsoft and dozens of other companies. Retrieved May 1, 2023 from https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610

[5] Davide Canali, Marco Cova, Giovanni Vigna, and Christopher Kruegel. 2011. Prophiler: A fast filter for the large-scale detection of malicious web pages. In *Proceedings of the 20th International Conference on World Wide Web*, 197–206.

[6] Alan Cao and Brendan Dolan-Gavitt. 2022. What the fork? Finding and analyzing malware in GitHub forks. In *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web*, 1–8.

[7] Justin Cappos, Justin Samuel, Scott Baker, and John H. Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM Conference on Computer and Communications Security*, 565–574.

[8] Catalin Cimpanu. 2018. Malware found in Arch Linux AUR package repository. Retrieved May 1, 2023 from https://www.bleepingcomputer.com/news/security/malware-found-in-arch-linux-aur-package-repository/
[9] Thomas Claburn. 2023. Python Package Index had one person on-call to hold back weekend malware rush. Retrieved May 1, 2023 from https://www.theregister.com/2023/05/22/python_package_index_on_call/
[10] Idan Digmi. 2023. The rising trend of malicious packages in open source ecosystems. Retrieved May 1, 2023 from https://snyk.io/blog/malicious-packages-open-source-ecosystems/
[11] NPM Docs. 2023. How npm handles the "scripts" field. Retrieved May 1, 2023 from https://docs.npmjs.com/cli/v9/using-npm/scripts
[12] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. In *Proceedings of 27th Annual Network and Distributed System Security Symposium*, 1–17.
[13] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.
[14] Yong Fang, Mingyu Xie, and Cheng Huang. 2021. Pbdt: Python backdoor detection model based on combined features. *Security and Communication Networks* 1 (2021), 1–13.
[15] Aurore Fass, Michael Backes, and Ben Stock. 2019. Jstap: A static pre-filter for malicious javascript detection. In *Proceedings of the 35th Annual Computer Security Applications Conference*, 257–269.
[16] Aurore Fass, Robert P. Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, 303–325.
[17] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. 2013. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering*, 752–761.
[18] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in NPM with a lightweight permission system. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*, 1334–1346.
[19] Python Software Foundation. 2003. Python package index. Retrieved May 1, 2023 from https://pypi.org/
[20] The Linux Foundation. 2020. Open source software supply chain security. Retrieved May 1, 2023 from https://project.linuxfoundation.org/hubfs/Reports/oss_supply_chain_security.pdf?hsLang=en
[21] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*, 13–16.
[22] Gartner. 2023. Gartner identifies top security and risk management trends for 2022. Retrieved May 1, 2023 from https://www.gartner.com/en/newsroom/press-releases/2022-03-07-gartner-identifies-top-security-and-risk-management-trends-for-2022
[23] David Gilbertson. 2018. I'm harvesting credit card numbers and passwords from your site. Here's how. Retrieved May 1, 2023 from https://david-gilbertson.medium.com/im-harvesting-credit-card-numbers-and-passwords-from-your-site-here-s-how-9a8cb347c5b5
[24] GitHub. 2020. NPM. Retrieved May 1, 2023 from https://www.npmjs.com/
[25] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2021. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. *Journal of Systems and Software* 172 (2021), 1–14.
[26] Danielle Gonzalez, Thomas Zimmermann, Patrice Godefroid, and Max Schäfer. 2021. Anomalicious: Automated detection of anomalous and potentially malicious commits on GitHub. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice*, 258–267.
[27] Raman Goyal, Gabriel Ferreira, Christian Kästner, and James Herbsleb. 2018. Identifying unusual commits on GitHub. *Journal of Software: Evolution and Process* 30, 1 (2018).
[28] Yacong Gu, Lingyun Ying, Huajun Chai, Chu Qiao, Haixin Duan, and Xing Gao. 2023. Continuous intrusion: Characterizing the security of continuous integration services. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*, 1561–1577.
[29] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2022. Investigating package related security threats in software registries. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*, 1151–1168.
[30] Eric Holmes. 2018. How I gained commit access to Homebrew in 30 minutes. Retrieved May 1, 2023 from https://medium.com/@vesirin/how-i-gained-commit-access-to-homebrew-in-30-minutes-2ae314df03ab
[31] Yangyu Hu, Haoyu Wang, Ren He, Li Li, Gareth Tyson, Ignacio Castro, Yao Guo, Lei Wu, and Guoai Xu. 2020. Mobile app squatting. In *Proceedings of the Web Conference 2020*, 1727–1738.

[32] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[33] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical detection of malicious NPM packages based on graph-based behavior modeling and matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.

[34] Dustin Ingram. 2023. In 2022, the @pypi team removed >12,000 unique projects (mostly malware). Retrieved May 1, 2023 from https://twitter.com/di_codes/status/1610781657128108033

[35] Jossef Harush Kadouri. 2023. Who broke NPM?: Malicious packages flood leading to denial of service. Retrieved May 1, 2023 from https://medium.com/checkmarx-security/who-broke-npm-malicious-packages-flood-leading-to-denial-of-service-77ac707ddbf1

[36] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and PyPi registries. In *Proceedings of the 2nd International Workshop on Deployable Machine Learning for Security Defense*, 132–156.

[37] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171–4186.

[38] Mohammad Taha Khan, Xiang Huo, Zhou Li, and Chris Kanich. 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*, 135–150.

[39] Swati Khandelwal. 2018. Password-guessing was used to hack Gentoo Linux Github account. Retrieved May 1, 2023 from https://thehackernews.com/2018/07/github-hacking-gentoo-linux.html

[40] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 569–586.

[41] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the security of Github {CI} workflows. In *Proceedings of the 31st USENIX Security Symposium*, 2747–2763.

[42] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of attacks on open-source software supply chains. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*, 1509–1526.

[43] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the feasibility of cross-language detection of malicious packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference*, 71–82.

[44] Ravie Lakshmanan. 2022. Malware strains targeting Python and JavaScript developers through official repositories. Retrieved May 30, 2023 from https://thehackernews.com/2022/12/malware-strains-targeting-python-and.html

[45] Ravie Lakshmanan. 2023. Python developers beware: Clipper malware found in 450+ PyPI packages! Retrieved May 1, 2023 from https://thehackernews.com/2023/02/python-developers-beware-clipper.html

[46] Chris Lamb and Stefano Zacchiroli. 2021. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software* 39, 2 (2021), 62–70.

[47] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious packages lurking in user-friendly Python package index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*, 606–613.

[48] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*, 672–684.

[49] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022. Exploring the unchartered space of container registry typosquatting. In *Proceedings of the 31st USENIX Security Symposium*, 35–51.

[50] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. arXiv:1907.11692. Retrieved from https://arxiv.org/abs/1907.11692

[51] Microsoft. 2020. OSS detect backdoor. Retrieved May 30, 2023 from https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor

[52] Anders Møller. 2020. Jelly: JavaScript/TypeScript static analyzer for call graph construction, library usage pattern matching, and vulnerability exposure analysis. Retrieved May 1, 2023 from https://github.com/cs-au-dk/jelly

[53] Anders Møller, Benjamin Barslev Nielsen, and Martin Toldam Torp. 2020. Detecting locations in JavaScript programs affected by breaking library changes. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–25.

[54] Phil Muncaster. 2023. Hundreds malicious packages NPM. Retrieved May 30, 2023 from https://www.infosecurity-magazine.com/news/hundreds-malicious-packages-npm/
[55] Benjamin Barslev Nielsen, Martin Toldam Torp, and Anders Møller. 2021. Modular call graph construction for security scanning of Node. js applications. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 29–41.
[56] NPM. 2019. Plot to steal cryptocurrency foiled by the npm security team. Retrieved May 1, 2023 from https://blog.npmjs.org/post/185397814280/plot-to-steal-cryptocurrency-foiled-by-the-npm
[57] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*, 1–10.
[58] Marc Ohm, Lukas Kempf, Felix Boes, and Michael Meier. 2020. Supporting the detection of software supply chain attacks through unsupervised signature generation. arXiv:2011.02235. Retrieved from https://arxiv.org/abs/2011.02235
[59] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, 23–43.
[60] Marc Ohm, Arnold Sykosch, and Michael Meier. 2020. Towards detection of software supply chain attacks by forensic artifacts. In *Proceedings of the 15th International Conference on Availability, Reliability and Security*, 1–6.
[61] Phylum. 2023. Phylum identifies 137 malicious npm packages. Retrieved May 1, 2023 from https://blog.phylum.io/phylum-identifies-98-malicious-npm-packages
[62] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*, 449–460.
[63] Gede Artha Azriadi Prana, Abhishek Sharma, Lwin Khin Shar, Darius Foo, Andrew E. Santosa, Asankhaya Sharma, and David Lo. 2021. Out of sight, out of mind? How vulnerable dependencies affect open-source projects. *Empirical Software Engineering* 26 (2021), 1–34.
[64] python. 2023. JavaScript private properties. Retrieved May 1, 2023 from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_properties
[65] PyTorch. 2022. Attackers target packages in multiple programming languages in recent software supply chain attacks, 2022. Retrieved May 1, 2023 from https://checkmarx.com/blog/attackers-target-packages-in-multiple-programming-languages-in-recent-software-supply-chain-attacks/
[66] PyTorch. 2022. Compromised PyTorch-nightly dependency chain between December 25th and December 30th, 2022. Retrieved May 1, 2023 from https://pytorch.org/blog/compromised-nightly-dependency/
[67] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
[68] Jukka Ruohonen, Kalle Hjerppe, and Kalle Rindell. 2021. A large-scale security-oriented static analysis of Python packages in PyPI. In *Proceedings of the 18th International Conference on Privacy, Security and Trust*, 1–10.
[69] Vitalis Salis, Thodoris Sotiropoulos, Panos Louridas, Diomidis Spinellis, and Dimitris Mitropoulos. 2021. Pycg: Practical call graph generation in python. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*, 1646–1657.
[70] Adriana Sejfia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*, 1681–1692.
[71] Ax Sharma. 2022. More than 200 cryptomining packages flood npm and PyPI registry. Retrieved May 1, 2023 from https://blog.sonatype.com/more-than-200-cryptominers-flood-npm-and-pypi-registry
[72] Snyk. 2022. The 2022 state of open source security report. Retrieved May 1, 2023 from https://go.snyk.io/state-of-open-source-security-report-2022.html
[73] Sonatype. 2023. Malware monthly - December 2022. Retrieved May 1, 2023 from https://blog.sonatype.com/malware-monthly-december-2022
[74] sonatype. 2023. State of the software supply chain. Retrieved May 1, 2023 from https://www.sonatype.com/state-of-the-software-supply-chain/introduction
[75] Ayrton Sparling. 2018. I don't know what to say. Retrieved May 1, 2023 from https://github.com/dominictarr/event-stream/issues/116
[76] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating deep code behaviors with metadata features for malicious PyPI Package detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.
[77] Synopsys. 2023. Open source security and risk analysis report. Retrieved May 1, 2023 from https://www.synopsys.com/content/dam/synopsys/sig-assets/reports/rep-ossra-2023.pdf

[78] Janos Szurdi, Balazs Kocso, Gabor Cseh, Jonathan Spring, Mark Felegyhazi, and Chris Kanich. 2014. The long "taile" of typosquatting domain names. In *Proceedings of the 23rd USENIX Security Symposium*, 191–206.
[79] Rashid Tahir, Ali Raza, Faizan Ahmad, Jehangir Kazi, Fareed Zaffar, Chris Kanich, and Matthew Caesar. 2018. It's all in the name: Why some URLs are more vulnerable to typosquatting. In *Proceedings of the 2018 IEEE Conference on Computer Communications*, 2618–2626.
[80] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*, 112–131.
[81] tree sitter. 2023. Tree-sitter: An incremental parsing system for programming tools. Retrieved May 1, 2023 from https://tree-sitter.github.io/tree-sitter/
[82] virustotal. 2023. Virustotal. Retrieved May 1, 2023 from https://www.virustotal.com/gui/home/upload
[83] Duc-Ly Vu. 2020. Bandit4Mal. Retrieved May 30, 2023 from https://github.com/lyvd/bandit4mal
[84] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. Lastpymile: Identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 780–792.
[85] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad snakes: Understanding and improving Python package index malware scanning. In *Proceedings of the 45th International Conference on Software Engineering*, 499–511.
[86] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Towards using source code repositories to identify software supply chain attacks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2093–2095.
[87] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and combosquatting attacks on the python ecosystem. In *Proceedings of the 2020 IEEE European Symposium on Security and Privacy Workshops*, 509–514.
[88] Warehouse. 2020. Malware checks. Retrieved May 1, 2023 from https://warehouse.pypa.io/development/malware-checks.html
[89] Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An empirical study of malicious code in PyPI ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 166–177.
[90] Liang Wentao, Ling Xiang, Wu Jingzheng, Luo Tianyue, and Yanjun Wu. 2023. A needle is an outlier in a haystack: Hunting malicious PyPI packages with code clustering. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 307–318.
[91] David A. Wheeler. 2005. Countering trusting trust through diverse double-compiling. In *Proceedings of the 21st Annual Computer Security Applications Conference*, 33–48.
[92] Meiqiu Xu, Ying Wang, Shing-Chi Cheung, Hai Yu, and Zhiliang Zhu. 2022. Insight: Exploring cross-ecosystem vulnerability impacts. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.
[93] Yanfang Ye, Tao Li, Donald Adjeroh, and S. Sitharama Iyengar. 2017. A survey on malware detection using data mining techniques. *ACM Computing Surveys* 50, 3 (2017), 1–40.
[94] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the NPM supply chain? In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 331–340.
[95] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the NPM and rubygems dependency networks. *Empirical Software Engineering* 27, 5 (2022), Article 107, 1–45.
[96] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small world with high risks: A study of security threats in the npm ecosystem. In *Proceedings of the 28th USENIX Security Symposium*, 995–1010.

Received 27 January 2024; revised 7 October 2024; accepted 24 October 2024